

Real Estate Agency Portal

By: Nasir Karim – CIS 344

Introduction

My project is a Real Estate Agency Portal, a web app where users can browse properties, view details, contact agents, and buy or rent a place. The main goal was to connect a PHP website to a MySQL database so everything works together smoothly and data is stored properly.

This system makes it easier to manage properties and user interactions in one place. It simulates how a real real estate website works, where users can search, view details, save favorites, send inquiries, and perform transactions. The app has different types of users like agents, buyers, and renters, and each one has different permissions. This makes the system more realistic and organized.

System Features

User System

Users can sign up and log in to the system. During registration, they choose a role such as agent, buyer, or renter. Each role controls what the user is allowed to do.

Passwords are not stored directly. Instead, they are hashed using password_hash, which makes the system more secure. When logging in, the system uses password_verify to check the password.

Role Based Access

Not all users can do the same actions, which keeps the system organized.

- Agents can add new properties, edit them, and delete properties they manage
- Buyers and renters can browse properties, save favorites, send inquiries, and perform transactions

This separation makes sure users only see and do what they are supposed to.

Property Management

Agents can add properties by entering details like:

- Title
- Property type

- Address
- City
- Price
- Images

Each property can also have two images (for example, interior and exterior), which improves how it looks to users. Once added, the property appears on the listings page. Users can click on any property to see full details, including all the information and images.

Search Feature

Users can search for properties using filters such as:

- City
- Minimum price
- Maximum price

This feature helps users find properties faster instead of scrolling through everything. It makes the system more user friendly.

Favorites System

Users can save properties they are interested in. These saved properties appear on a separate Favorites page. This is useful because users don't have to search again if they want to revisit a property later.

Inquiry System

Users can send messages about a property. For example, they can ask questions or request more details. These inquiries are saved in the database and linked to both the user and the property. This allows agents to later see who is interested in their properties.

Transactions: Buy and Rent

Users can buy or rent a property directly from the system. When this happens:

- A transaction is saved in the database
- The property status changes to sold or rented
- The property is no longer available for other users

This makes the system behave more like a real world application.

Database Design

The system uses a relational database with multiple tables that are connected together.

Main Tables

- Users – stores user information like username, password (hashed), role, and contact info
- Properties – stores all property details
- Transactions – stores records of buying and renting
- Inquiries – stores messages sent by users
- Favorites – stores saved properties

Each table is connected using foreign keys. This helps keep the data consistent and prevents errors.

Stored Procedure, Trigger, and View

To make the system more advanced and efficient, I used database features like a stored procedure, a trigger, and a view.

Stored Procedure

I created a stored procedure called ProcessTransaction. This procedure:

- Inserts a transaction into the database
- Updates the property status automatically

This makes the system cleaner because the logic is handled inside the database instead of repeating code in PHP.

Trigger

I also used a trigger that runs automatically after a transaction is inserted. The trigger updates the property status based on whether the property was bought or rented. This helps keep the database accurate without needing extra manual updates.

View

I used a database view to simplify property listing queries. The view combines data from multiple tables, like properties and agents, so it is easier to display property listings in a clean and simple way without writing long joins in every query.

How the System Works

The system follows a simple flow:

1. A user registers and logs in
2. The user browses available properties
3. The user can:
 - Save properties to favorites
 - Send inquiries
 - Buy or rent a property
4. When a transaction happens:
 - The transaction is stored
 - The property status is updated
 - The property is no longer available
5. Agents can manage properties by adding, deleting, or editing them

Problems and Fixes

Favorites Not Updating Correctly

At first, the favorites system did not work correctly because some method names in the PHP files did not match the methods in the RealEstateData class. For example, the code was calling the old method name while the class had a new one. I fixed this by making sure all the method names in the files matched the ones defined in the class.

Property Details Buttons Showing at the Wrong Time

On the property details page, the Buy and Rent buttons were still showing even when the property was already sold or rented. The problem was that the page was not checking the property status properly. I fixed this by adding a condition that only shows the buttons if the property status is “available”.

Search Feature Ignoring Filters

Sometimes the search page showed all properties even when filters were entered. This happened because empty values were still being passed into the search function. I fixed it by checking if the user actually entered a city or price before adding those conditions to the SQL query.

Inquiry Form Not Saving Messages

The inquiry form was not saving messages at first. The issue was that the `propertyId` was not always passed correctly between GET and POST requests. I fixed this by checking both `$_GET["propertyId"]` and `$_POST["propertyId"]`, so the form always has the correct property ID.

Transaction Not Updating Property Status

When buying or renting a property, the transaction was saved, but sometimes the property status did not change. The issue was that the stored procedure was not being called correctly or with the right parameters. After fixing the method name and making sure the correct values were passed, the property status updated as expected.

Errors When Deleting Properties

When I tried to delete a property, I got foreign key errors because there were still related records in other tables like Favorites, Inquiries, and Transactions. I fixed this by deleting the related records first and then deleting the property. This respects the foreign key constraints and keeps the database clean.

Images Not Showing on Some Pages

On some pages, property images were not showing. The problem was that the image field names in the PHP code did not match the column names in the database (`image_url` and `image_url2`). I fixed this by using the correct column names everywhere in the project.

This project connects PHP and MySQL to build a working real estate system with features like login, role-based access, property listings, search, favorites, inquiries, and transactions.

Working on this project helped me understand:

- How backend systems work
- How databases connect to web applications
- How to use stored procedures, triggers, and views
- How to debug real problems when things don't work the first time

This project gave me good practice with real-world style web development and database design, not just simple examples. I still need to practice more though.